

Caveats

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

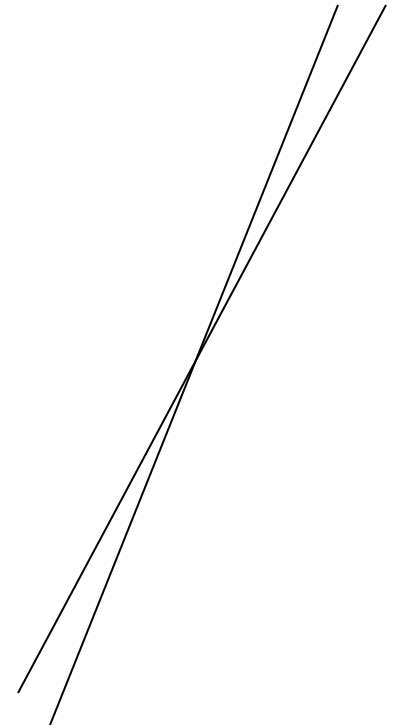


Floating Point Issues

- Floating point numbers like 1.0101×2^{-6} stored as a mantissa and exponent. Some numbers (e.g., $1/2$) exactly representable; others (e.g., $1/3$) are not.
- Roundoff errors inherent in floating point arithmetic.
 - E.g., multiplying two n -digit numbers gives a $2n$ -digit number
- Test equality of x and y by checking if $|x - y| \leq \epsilon$.
- Adding a large number to a small number is sometimes risky.
 - E.g., using a 32-bit float to add 2^{20} copies of 0.1 gives 105891.84375 instead of the intended answer 104857.6 (divide and conquer gives better result here).
- Subtracting two numbers agreeing in many initial digits might give a very low-precision result (“catastrophic cancelation”).

Poorly-Conditioned Inputs

- A specific input to a problem is poorly conditioned if small changes to the input might cause large changes in input.
- Example: finding the intersection point of two nearly-parallel lines.
- In this situation, roundoff errors can be more worrisome.



Stability

- Algorithm “unstable” if errors are compounded by feeding back into each subsequent step.
- Examples:
 - Euler’s method for recovering a function from its derivatives
 - Gram-Schmidt orthogonalization
 - The conjugate gradient method
- These algorithms are still used but should be used with care.

